

ReliefChain – Project Report

Oct 9, 2026 · @Ashutosh

Title page

ReliefChain: Blockchain-Based Disaster Relief Fund Tracker

A mini-project report on the syllabus topic *"Develop a Blockchain based application for transparent and genuine charity"*

Submitted by	_____ (Roll No. _____)
Guided by	_____
Department	_____
College	_____
Academic year	2026-27

Certificate

This is to certify that the mini-project titled **"ReliefChain: Blockchain-Based Disaster Relief Fund Tracker"** is the bonafide work of _____ (Roll No. _____), carried out under my guidance in partial fulfilment of the requirements of _____ during the academic year 2026-27.

Project guide	_____
Head of department	_____
Date	_____

Acknowledgement

We sincerely thank our project guide, _____, for continuous guidance and encouragement throughout this project. We also thank the Head of the Department and all faculty members for their support, and our classmates for their feedback during testing.

Finally, we thank the open-source communities behind Ethereum, Hardhat, OpenZeppelin, FastAPI and React, whose tools made this project possible.

Abstract

ReliefChain is a web application that records every disaster-relief donation and every use of the collected funds on an Ethereum blockchain, so that anyone can see and verify them. Donors usually cannot check whether their donation was counted or how it was spent, because charity records sit in private databases that can be edited.

In ReliefChain, a donor enters a name and an amount in rupees. A FastAPI backend writes the donation to a Solidity smart contract, which stores it permanently and returns a transaction hash as the donor's receipt. The relief organization records allocations by category (Food, Medicine, Shelter, Transportation, Other), and the contract rejects any allocation larger than the funds actually donated. A public transparency dashboard shows raised, allocated and remaining funds, and a verification page fetches any transaction directly from the blockchain.

The system runs on a local Hardhat blockchain with a React frontend. It passed 15 of 15 smart-contract tests, 5 of 5 backend integration tests and 51 of 51 automated browser checks. The project also states its limits clearly: blockchain makes recorded activity transparent and tamper-evident, but it does not by itself prove that an organization is genuine.

Keywords: blockchain, smart contract, Ethereum, Solidity, transparency, charity, disaster relief

Chapter 1: Introduction

1.1 Background

Floods, cyclones and earthquakes regularly displace families across India, and relief work depends heavily on public donations. People give generously after a disaster, but they rarely see what happens to their money afterwards. Trust in the relief organization is the only guarantee they have.

1.2 Motivation

When donation records are kept in an organization's own database, the organization can change or delete entries and nobody outside would know. Spending is usually published only as totals, often months later. Fake fundraisers also appear after major disasters, and a

donor cannot easily tell a genuine appeal from a fake one. This project explores how blockchain technology can make the financial record of a relief campaign public and verifiable.

1.3 Blockchain basics

A **blockchain** is a shared digital ledger made of blocks. Each block holds a list of transactions and the cryptographic hash of the previous block, so the blocks form a chain. Changing an old block changes its hash and breaks the chain, which makes tampering easy to detect.

- **Ethereum** is a public blockchain that can run programs called smart contracts. Fees for running transactions are called gas.
- A **smart contract** is a program stored on the blockchain that runs exactly as written. Its rules cannot be changed after deployment.
- **Solidity** is the language used to write Ethereum smart contracts.
- A **transaction hash** is a unique 66-character ID for every transaction. Anyone can use it to look up the transaction.
- A **wallet address** is the public identity of an account. Its matching private key signs transactions.

1.4 About this project

ReliefChain applies these ideas to disaster relief. Every donation and every fund allocation becomes a blockchain transaction with its own hash, and the smart contract enforces rules such as "spending can never exceed donations". The project is a working prototype on a local test blockchain, built as a college mini-project.

Chapter 2: Literature Survey and Existing System

2.1 Existing system

Most charities record donations in their own database or spreadsheets and publish an annual report. Donors receive an email or PDF receipt. This works, but it leaves the donor with no independent way to check the records.

Aspect	Existing system	Weakness
Record keeping	Private database controlled by the charity	Entries can be changed or deleted silently

Aspect	Existing system	Weakness
Reporting	Annual or periodic report	Not real-time; spending shown only as totals
Donor receipt	Email or PDF	Cannot be independently verified
Use of funds	Described in text, if at all	No category-wise trail a donor can check
Fake appeals	Hard to distinguish from genuine ones	No public record to compare against

2.2 Blockchain in real-world aid and charity

Several organizations already use blockchain to make aid more transparent.

Initiative	Organization	What it does
Building Blocks	UN World Food Programme	First field test in January 2017 in Sindh Province, Pakistan; now a shared blockchain network where aid organizations coordinate assistance and avoid duplication. WFP reports support for 4.8 million households, with operations including Jordan, Bangladesh and Ukraine.
CryptoFund	UNICEF	Launched 9 October 2019 as the first UN fund to receive, hold and disburse donations in cryptocurrency (ether and bitcoin), granted out in the same currency to open-source projects.
GiveTrack	BitGive Foundation	Beta launched in October 2017; lets donors give bitcoin and follow the flow of funds from donation to allocation on a public ledger, with project updates.

2.3 Gap addressed by this project

These systems are large and built for international agencies or cryptocurrency donors. ReliefChain is a simplified model of the same idea for a campus-level prototype. Donors need no crypto wallet, amounts are in rupees, and the smart contract itself enforces that recorded spending can never exceed recorded donations.

Chapter 3: Problem Statement, Objectives and Scope

3.1 Problem statement

Donors to disaster-relief campaigns cannot independently verify that their donation was recorded, how much a campaign collected, or how the money was allocated. The records are controlled by one party and can be changed without detection. The project must provide a tamper-evident, publicly verifiable record of donations and fund allocations.

3.2 Objectives

1. Record every donation on a blockchain with a unique transaction hash.
2. Record how collected funds are allocated, by category: Food, Medicine, Shelter, Transportation and Other.
3. Prevent recording allocations larger than the available funds, enforced by the smart contract.
4. Restrict all write actions to the relief organization's wallet.
5. Show a public transparency dashboard of raised, allocated and remaining funds.
6. Let anyone verify a transaction directly from the blockchain.
7. Keep the system simple enough to demonstrate in about five minutes.

3.3 Scope

In scope: campaign creation, donations recorded in rupees, fund allocation, closing campaigns, automatic campaign status, donation history, transparency dashboard, transaction verification and an admin panel.

Out of scope: real payment collection (a payment gateway), verifying that an organization is genuine, deployment on a public blockchain, user accounts and complex authentication. All campaigns and donors are fictional demonstration data.

Chapter 4: System Requirements

The project runs entirely on one laptop; no paid services or real cryptocurrency are needed.

4.1 Hardware requirements

Component	Minimum
Processor	Dual-core, 2 GHz or better

Component	Minimum
RAM	4 GB (8 GB recommended)
Disk space	2 GB free for dependencies
Network	Needed only to install packages

4.2 Software requirements

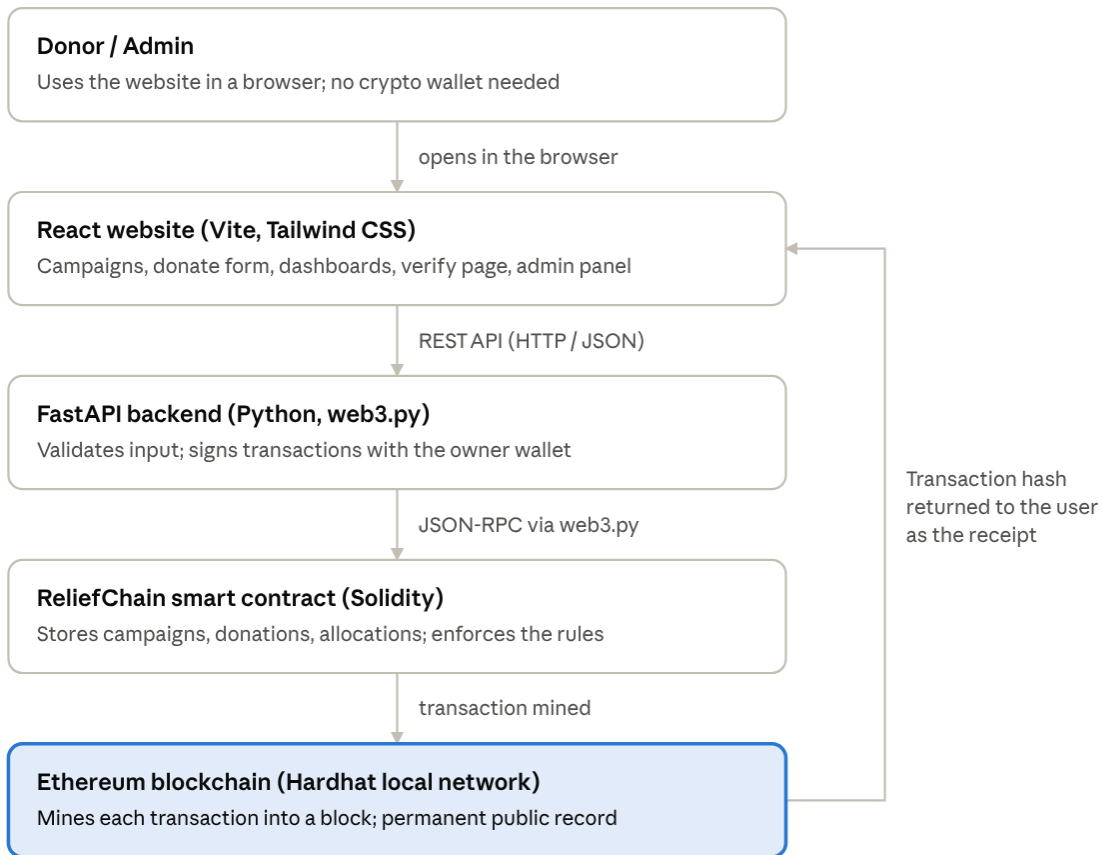
Software	Version used	Purpose
Operating system	Windows 11 (also runs on macOS / Linux)	Development and demo
Node.js and npm	Node 22.20, npm 10.9	Hardhat and the React frontend
Python	3.11	FastAPI backend
Solidity	0.8.24	Smart contract language
Hardhat	2.29	Local Ethereum blockchain, compile, test, deploy
OpenZeppelin Contracts	5.6	<code>Ownable</code> access control
FastAPI + Uvicorn	0.115 / 0.32	REST API server
web3.py	7.16	Python library to call the smart contract
React + Vite	React 19, Vite 8	Web frontend
Tailwind CSS	4.3	Styling
Recharts	3.10	Allocation donut chart
Web browser	Chrome / Edge	Using the application

Chapter 5: System Design

ReliefChain uses a five-layer architecture in which the smart contract is the single source of truth for all financial data.

5.1 Architecture

Every write passes the backend and the contract before the chain



ReliefChain system architecture · 5 layers

The user never talks to the blockchain directly: the backend holds the organization's wallet, which owns the contract, and sends every transaction on the user's behalf. Read-only requests (campaign lists, history, verification) follow the same path but create no transaction.

5.2 Donation flow

1. The donor opens a campaign, enters a name and an amount in rupees, and clicks Donate.
2. The website sends `POST /api/campaigns/{id}/donate` to the backend.
3. The backend validates the input and simulates the call; if a contract rule would fail, the reason is shown and nothing is sent.
4. The backend signs `donate(id, name, amount)` with the owner key and sends it to the blockchain.
5. The contract checks its rules, stores the donation, updates totals and emits `DonationReceived`.

6. The transaction is mined; the backend returns the transaction hash and block number, shown as the receipt.

Allocations follow the same path through `allocateFunds`, and the contract additionally checks that the amount does not exceed raised minus already-allocated funds.

5.3 Data storage design

Data	Stored in	Reason
Campaigns, targets, status	Smart contract	Must be public and tamper-evident
Donations (name, ₹, time)	Smart contract	Core financial record
Allocations (category, ₹, note)	Smart contract	Core financial record
Transaction hashes	Blockchain event logs	Proof that each record exists
Campaign images	Local files in <code>frontend/public/images</code>	Large files are expensive on a blockchain; only a short path is stored on-chain
Admin key, private key	<code>backend/.env</code>	Secrets are never put on-chain

No separate database (such as MySQL or MongoDB) is used.

5.4 User roles

Role	Can do	How access is controlled
Donor (anyone)	View campaigns, donate, see history, verify transactions	Open website
Admin (relief organization)	Create campaigns, allocate funds, close campaigns	Admin key in the website; <code>onlyOwner</code> in the contract

Chapter 6: Smart Contract Design and Security

The contract `ReliefChain.sol` (Solidity 0.8.24) is the single source of truth for all financial data. It inherits OpenZeppelin's `Ownable`, so the deployer (the relief organization's wallet) is the only account allowed to write.

6.1 Data structures

Structure	Fields
Campaign	id, name, disasterType, location, description, imageUrl, targetAmount, raisedAmount, allocatedAmount, deadline, createdAt, creator, status, donorCount, donationCount
Donation	donorName, amount (₹), timestamp
Allocation	category, amount (₹), note, timestamp
Status (enum)	Active, Completed, Closed
Category (enum)	Food, Medicine, Shelter, Transportation, Other

Campaigns are stored in a mapping from campaign ID to Campaign ; donations and allocations are arrays per campaign. A second mapping stores keccak256(donorName) per campaign to count unique donors with a fixed 32-byte key.

6.2 Functions

Function	Caller	Purpose
createCampaign(name, type, location, description, imageUrl, target, deadline)	Owner	Opens a new campaign with status Active
donate(campaignId, donorName, amount)	Owner (backend)	Records a donation; sets status to Completed when the target is reached
allocateFunds(campaignId, category, amount, note)	Owner	Records a use of funds by category
closeCampaign(campaignId)	Owner	Sets status to Closed; no more donations
getCampaign, getAllCampaigns, getDonations, getAllocations, getAvailableFunds	Anyone	Free read-only (view) functions

6.3 Events

The contract emits `CampaignCreated`, `DonationReceived`, `FundsAllocated` and `CampaignStatusChanged`. Events are a permanent, searchable log; the backend reads them to find the transaction hash of every donation and allocation.

6.4 Security checks

Check	Why it exists
<code>onlyOwner</code> on every write function	Outsiders cannot create fake campaigns, donations or allocations
<code>validCampaign</code> : ID must be between 1 and <code>campaignCount</code>	No record can be attached to a campaign that does not exist
Amount greater than 0; donor name 1–64 characters	No empty or meaningless records
Donations only when status is Active and before the deadline	Closed, completed or expired campaigns cannot keep collecting
Allocation amount must not exceed raised minus already allocated	The organization cannot claim to spend money it never received
<code>Category</code> is an enum	Invalid categories are rejected automatically
Solidity 0.8 built-in overflow checks	Arithmetic cannot silently wrap around

All checks use `require` statements with clear messages (for example "Allocation exceeds available funds"), which the backend passes on to the user.

Chapter 7: Implementation

The project has three parts in one folder: the Hardhat project (contract, tests, deploy script), the FastAPI backend and the React frontend.

7.1 Project structure

Path	Contents
<code>contracts/ReliefChain.sol</code>	Solidity smart contract
<code>scripts/deploy.js</code>	Deploys the contract, saves its address and ABI, seeds 4 demo campaigns

Path	Contents
<code>test/ReliefChain.test.js</code>	15 Hardhat tests
<code>backend/main.py</code>	FastAPI routes, input validation, admin key check
<code>backend/blockchain.py</code>	web3.py layer: send transactions, read data and events, verify hashes
<code>backend/test_api.py</code>	5 integration tests against the live blockchain
<code>frontend/src/components/</code>	Navbar, CampaignCard, DonationForm, DonationHistory, FundAllocation, AllocationChart, TransparencyDashboard, TransactionVerification, AdminPanel, BlockchainStatus, ProgressBar, StatsCard, StatusBadge, TxHash, Footer
<code>frontend/src/pages/</code>	Home, Campaigns, CampaignPage, Dashboard, Transparency, Verify, Admin, About

7.2 Backend (FastAPI + web3.py)

The backend holds the private key of the contract owner (Hardhat test account #0) in `backend/.env`. For every write it first runs `estimate_gas`, which simulates the call: if a `require` would fail, the reason is returned to the user and no transaction is sent. Otherwise it signs the transaction, sends it, waits for it to be mined and returns the transaction hash, block number and gas used. A lock ensures transactions are sent one at a time with correct nonces.

Endpoint	Method	Purpose
<code>/api/health</code>	GET	Blockchain connection, block number, contract address, owner check
<code>/api/campaigns</code>	GET	All campaigns with computed progress and status
<code>/api/campaigns/{id}</code>	GET	One campaign with donations and allocations, each with its tx hash
<code>/api/campaigns/{id}/donate</code>	POST	Record a donation (name, amount in ₹)
<code>/api/stats</code>	GET	Platform totals, allocation by category, recent transactions

Endpoint	Method	Purpose
<code>/api/verify/{txHash}</code>	GET	Look up and decode a transaction from the blockchain
<code>/api/admin/login</code>	POST	Check the admin key
<code>/api/admin/campaigns</code>	POST	Create a campaign (admin key required)
<code>/api/admin/campaigns/{id}/allocate</code>	POST	Record an allocation (admin key required)
<code>/api/admin/campaigns/{id}/close</code>	POST	Close a campaign (admin key required)

Input is validated with Pydantic, and every error (blockchain offline, contract not deployed, invalid campaign, rejected rule) is returned as a short, friendly message.

7.3 Frontend (React + Vite + Tailwind CSS)

The website is a single-page React app. It calls the backend through a Vite proxy and never holds any key. Key screens:

- **Campaigns** with search and status filters, progress bars and days left.
- **Campaign page** with target, raised, remaining, donors, the donation form, the allocation donut chart and the donation history.
- **Transparency** page with totals, a category chart, a per-campaign breakdown and a public ledger of transactions.
- **Verify** page that decodes any transaction hash.
- **Admin** panel to create campaigns, allocate funds and close campaigns.

Every page has loading, empty and error states, works at phone width, and shows a live blockchain connection indicator.

7.4 Deployment

`npx hardhat node` starts a local blockchain at `127.0.0.1:8545` (chain ID 31337). `npx hardhat run scripts/deploy.js --network localhost` deploys the contract, writes `backend/contract/ReliefChain.json` (address + ABI) and creates four fictional demo campaigns with real donation and allocation transactions. The backend reloads the contract automatically after each deploy.

Chapter 8: Testing and Results

All 71 automated checks passed: 15 of 15 smart-contract tests, 5 of 5 backend integration tests and 51 of 51 browser checks.

8.1 Smart-contract tests (Hardhat, Mocha + Chai)

No.	Test case	Expected result	Result
1	Contract deployment	Deployer is owner; zero campaigns	Pass
2	Campaign creation	Campaign stored, CampaignCreated emitted	Pass
3	Invalid campaign input	Empty name, zero target, past deadline rejected	Pass
4	Single donation	Raised amount, donor count and history updated	Pass
5	Invalid donation	Zero amount and empty name rejected	Pass
6	Donation after deadline	Rejected with "Campaign has ended"	Pass
7	Multiple donations	Totals add up; repeat donor counted once	Pass
8	Target reached	Status becomes Completed; further donations rejected	Pass
9	Fund allocation	Allocations stored; available funds reduced	Pass
10	Unauthorized admin action	Non-owner blocked from all write functions	Pass
11	Invalid campaign ID	IDs 0 and 99 rejected everywhere	Pass
12	Over-allocation	Allocation above available funds rejected	Pass
13	Invalid allocation	Zero amount and invalid category rejected	Pass
14	Campaign closing	Status Closed; donations blocked; remaining funds still allocatable	Pass
15	Starts empty	getAllCampaigns returns no campaigns	Pass

8.2 Backend integration tests (pytest, live blockchain)

No.	Test case	Result
1	Backend wallet is the contract owner	Pass

No.	Test case	Result
2	Donation → history shows the same tx hash → verification confirms ₹500	Pass
3	Allocation succeeds; over-allocation returns "Allocation exceeds available funds"	Pass
4	Completed and Closed status; donation to a closed campaign rejected	Pass
5	Missing admin key, invalid campaign, invalid input, fake and malformed hashes rejected	Pass

8.3 Browser tests (51 checks in Chrome)

An automated script clicked through the real website like a user and compared every number on screen with the smart contract. It covered the home page totals, search and filters, campaign details, the donation form (including empty and zero-amount errors), the donation receipt and history, verification of real, tampered and malformed hashes, admin login, allocation, over-allocation, campaign creation and closing, the transparency and dashboard totals, error pages, phone-width layout on 8 pages and the absence of JavaScript errors.

8.4 Issues found and fixed during testing

- A scroll helper returned a value from a React effect and crashed every page; fixed.
- A Tailwind CSS v4 rule broke the frontend build; custom styles were converted to `@utility` rules.
- The contract hit Solidity's "stack too deep" limit; string parameters were moved from `calldata` to `memory`.
- Entering ₹0 showed the browser's own popup instead of the app's message; the form now uses the app's validation.

Chapter 9: Output Screens

All screens below were captured from the running application with fictional demo data on the local Hardhat blockchain.

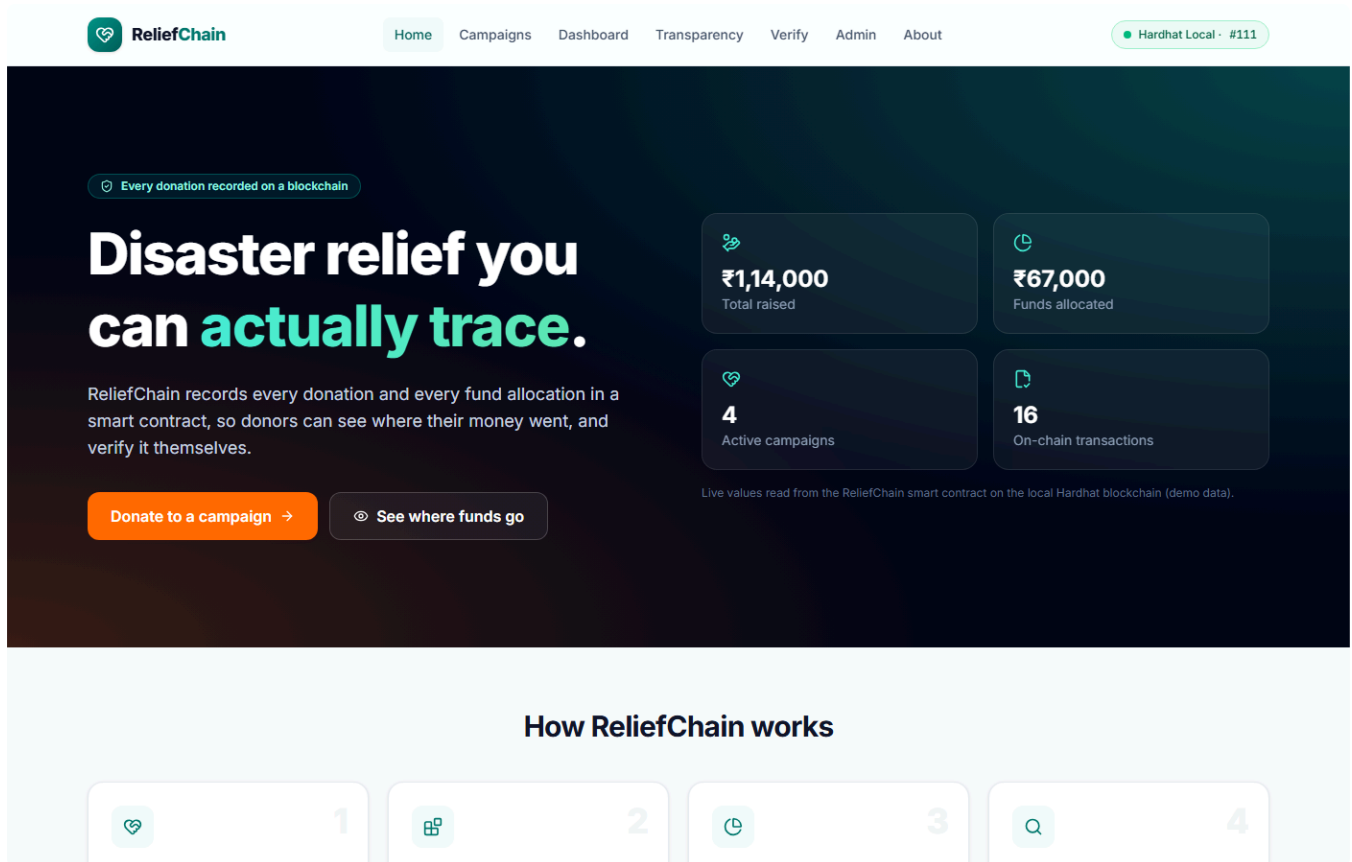


Figure 9.1: Home page. Live totals are read from the smart contract.

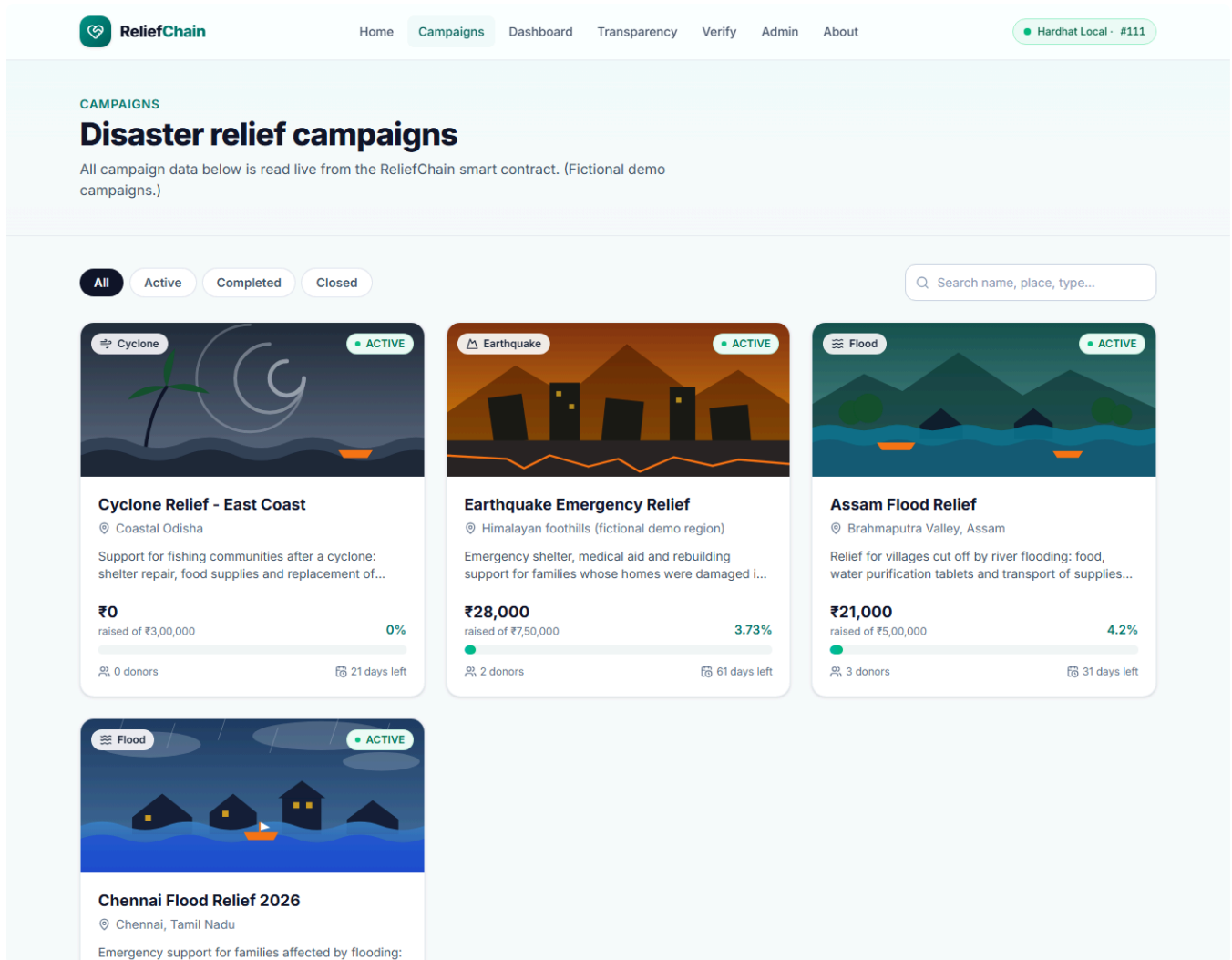


Figure 9.2: Campaign list with status filters, search and progress bars.

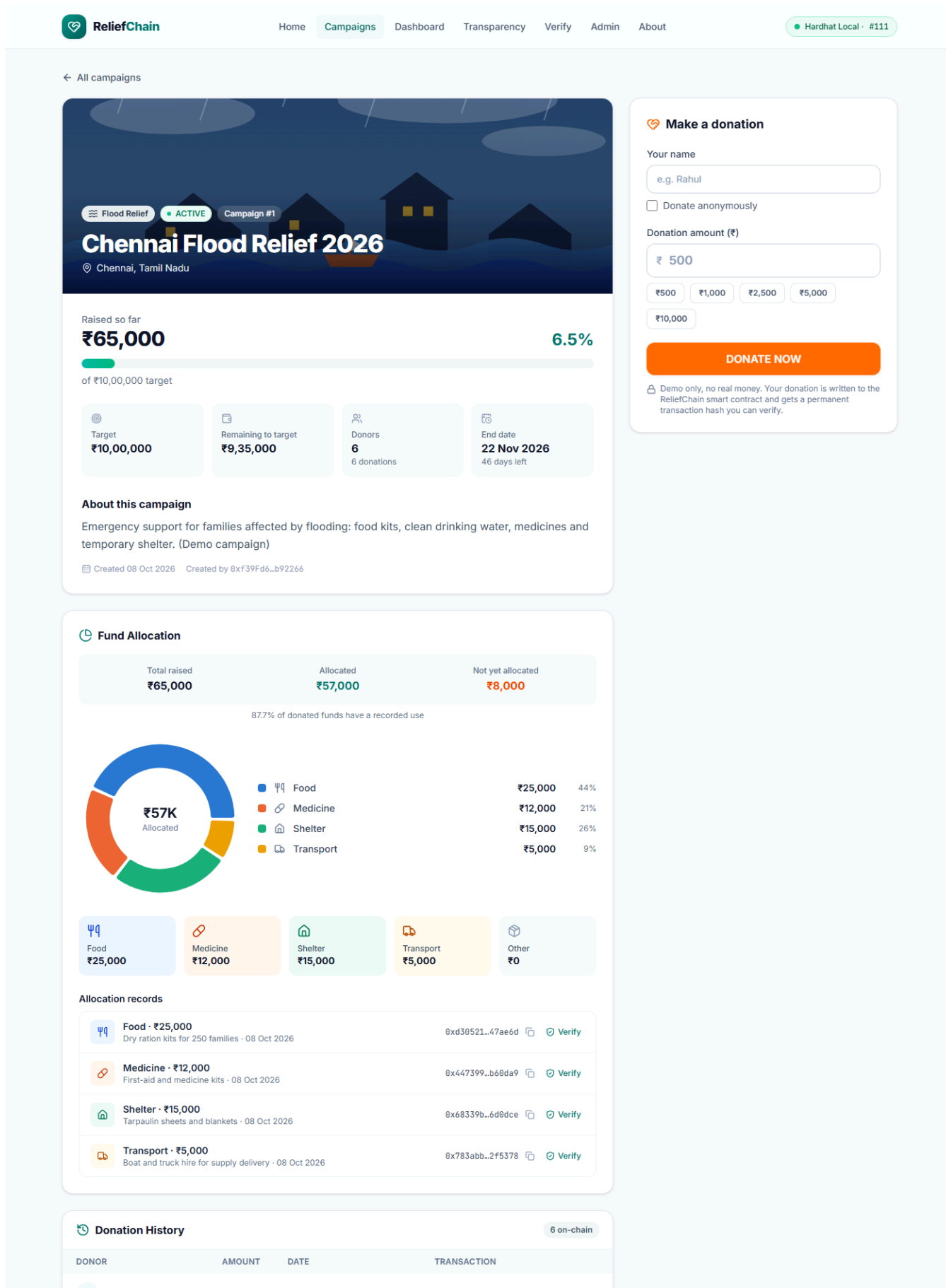


Figure 9.3: Campaign page showing target, raised, progress, donors, the donation form, fund allocation and donation history.

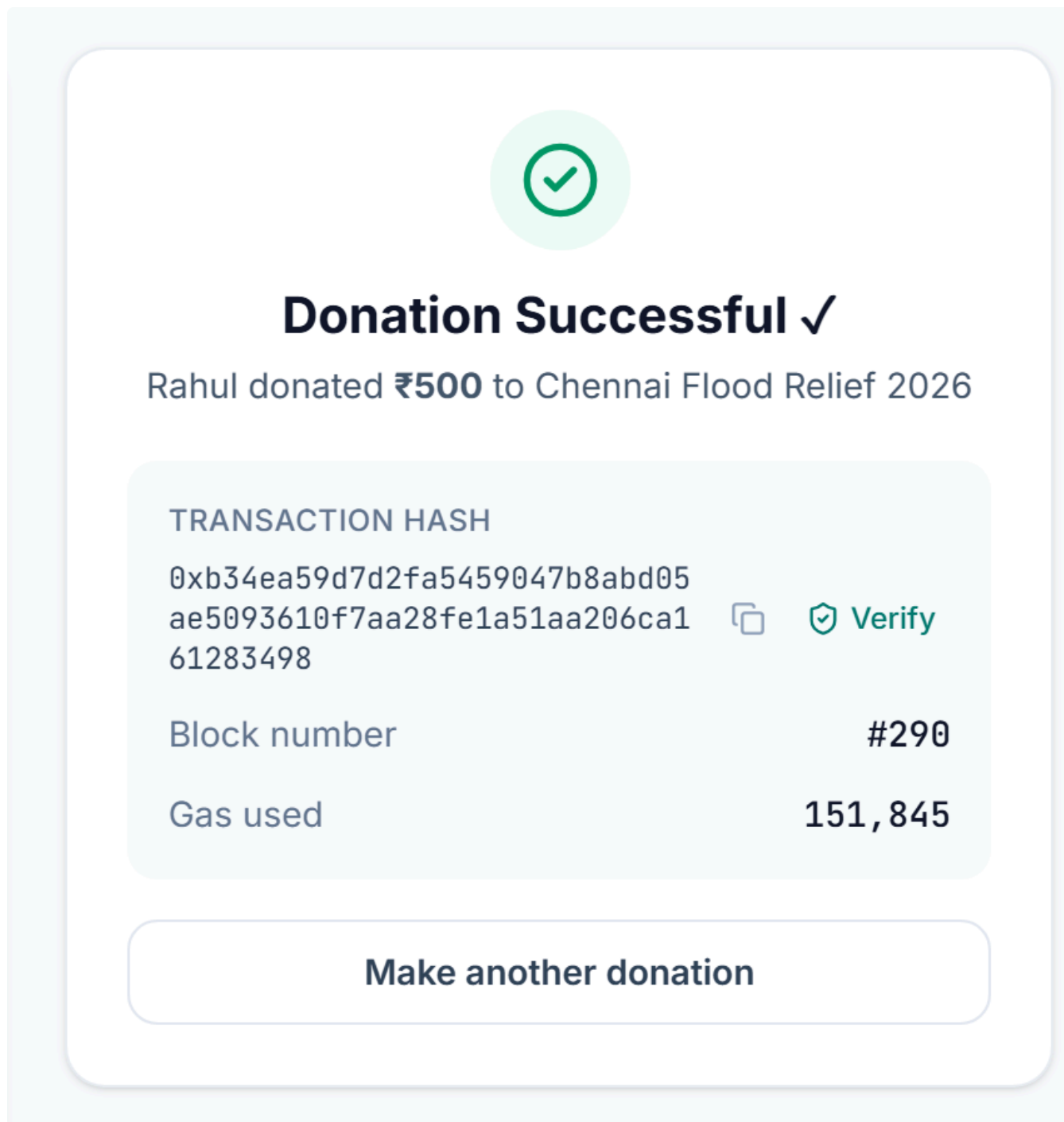


Figure 9.4: Donation receipt. The transaction hash and block number come from the mined blockchain transaction.

Record a fund allocation

The smart contract rejects any allocation larger than the campaign's unallocated funds.

Campaign

#1 Chennai Flood Relief 2026 (₹3,500 available)

Raised
₹65,500

Allocated
₹62,000

Available
₹3,500

Category

🍲 Food

🩹 Medicine

🏠 Shelter

🚚 Transport

📦 Other

Amount (₹)

500

Note (what was it used for?)

ORS packets

Use all available (₹3,500)

⌚ Allocate funds

Figure 9.5: Admin fund allocation. The smart contract rejects any amount larger than the available funds.

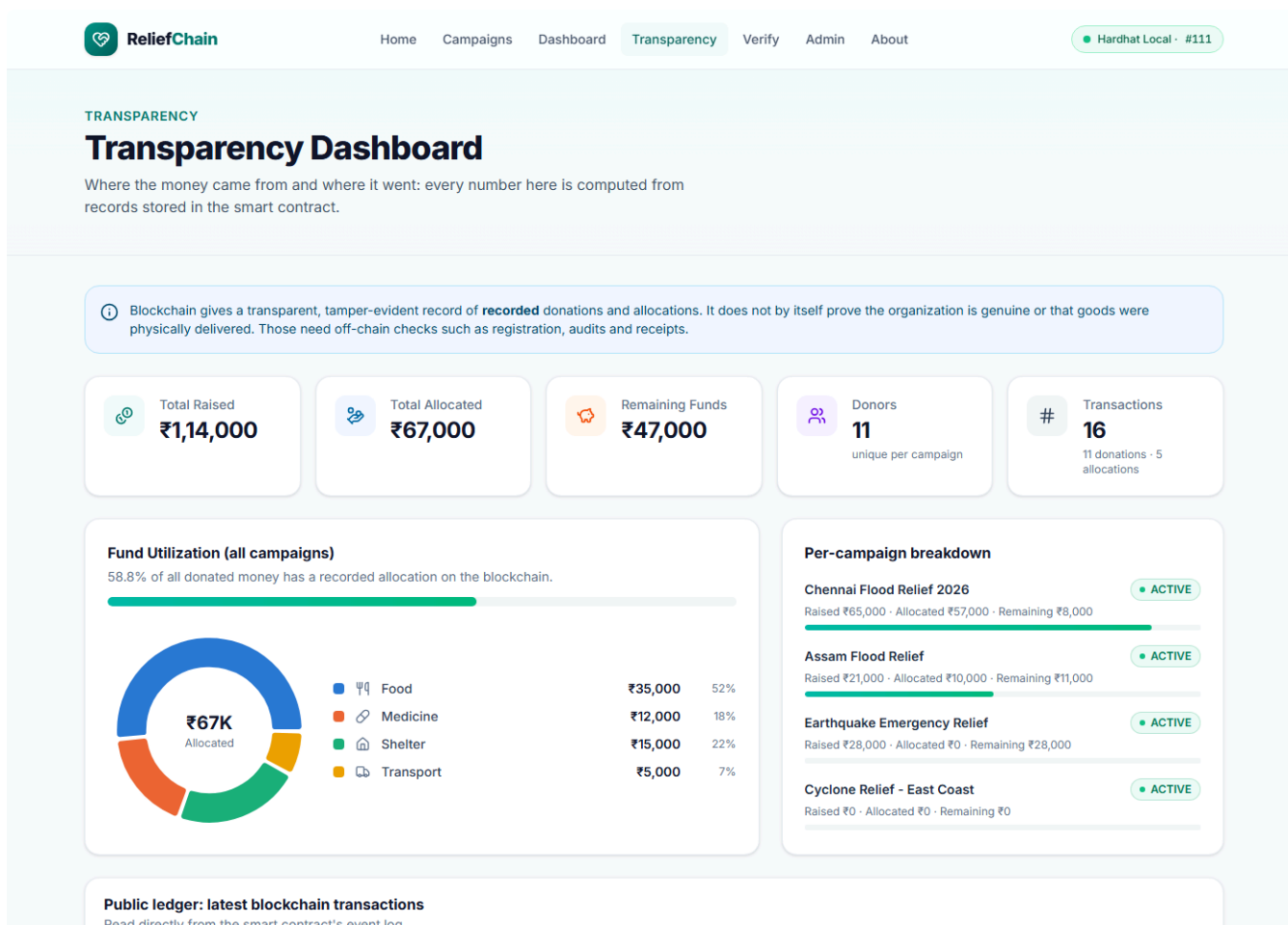


Figure 9.6: Transparency dashboard with totals, the allocation donut chart, a per-campaign breakdown and the public ledger.

 ReliefChain

HomeCampaignsDashboardTransparencyVerifyAdminAbout

● Hardhat Local · #111

VERIFICATION

Verify a blockchain transaction

Enter a transaction hash. ReliefChain fetches it straight from the blockchain and decodes what the smart contract recorded.

Transaction hash

0xa41dc55fc59ec71edc1650dd08a177abb8184cbbe917c55bc47015b34987b

Q VERIFY TRANSACTION

Tip: click **Verify** next to any donation or allocation, or paste a hash from a donation receipt.

 **✓ Transaction Found**

Fetches live from the blockchain, not from a database.

DONATION RECORD

Campaign

Chennai Flood Relief 2026 #1

Donor

Sneha Reddy

Donation amount

₹7,500

Status

Confirmed (14 confirmations)

Transaction hash

0xa41dc55fc59ec71edc1650dd08a177abb8184cbbe917c55bc47015b34987b1aa 

Block number

#98

Timestamp

08 Oct 2026, 11:33 pm

Sent by (recording wallet)

0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266

Sent to (contract)

0x202Cce504e04bEd6fC0521238d0f04Bc9E8E15a8

Gas used

151,917

Figure 9.7: Transaction verification. The record is fetched and decoded directly from the blockchain.

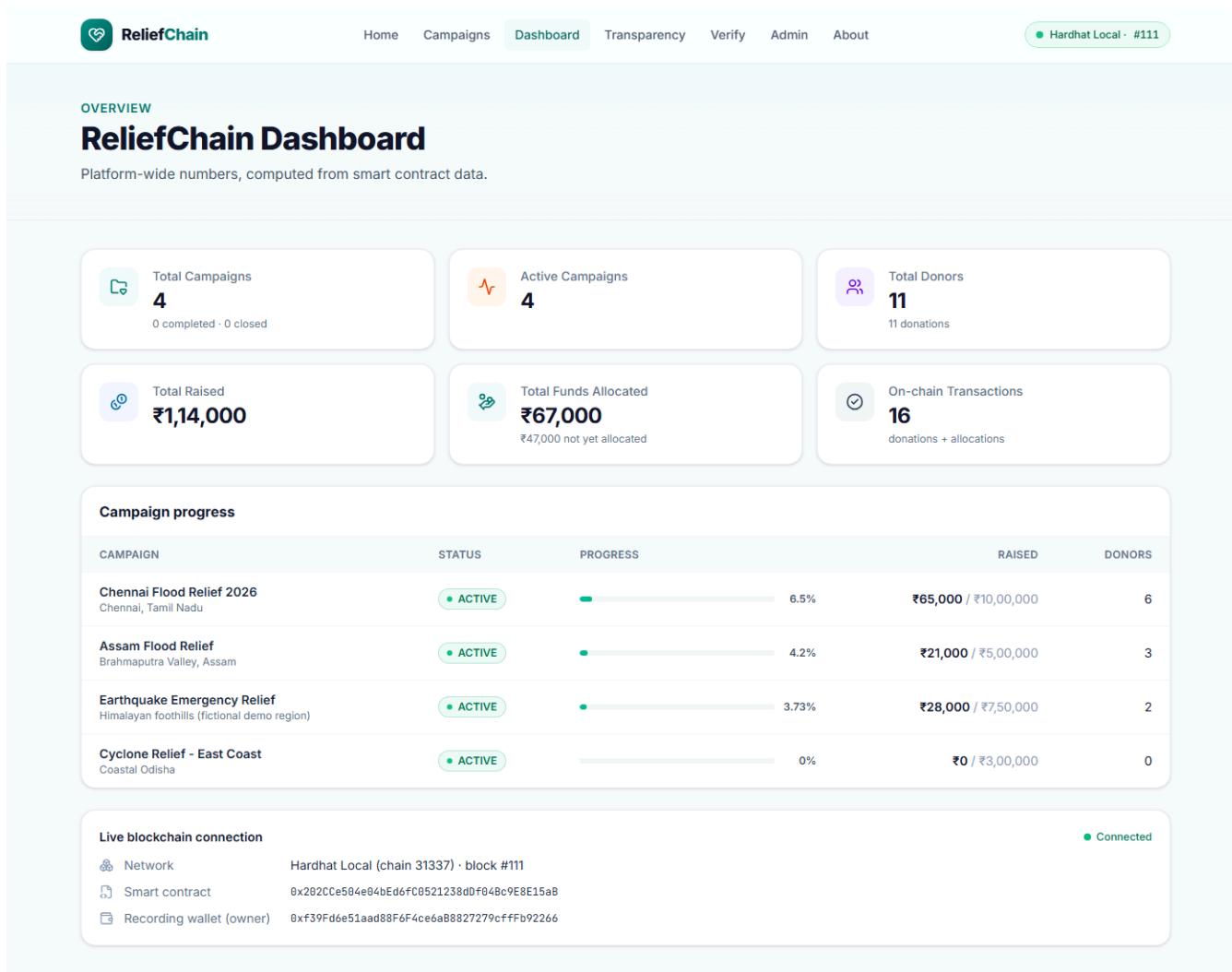


Figure 9.8: Project dashboard with campaign counts, donors, raised and allocated funds, and the live blockchain connection.

Chapter 10: Real-Life Applications

The same design works wherever people donate money and want proof of how it was used.

10.1 Where ReliefChain can be used

Area	How it would be used
Disaster-relief NGOs	Flood, cyclone and earthquake funds publish every donation and every rupee spent, by category
Government relief funds	State relief funds show citizens how disaster money is distributed

Area	How it would be used
Corporate CSR	Companies show where their CSR spending went, project by project
Medical crowdfunding	Families raising money for treatment show donors how hospital bills were paid
Temple and trust donations	Large trusts make offerings and their use publicly auditable
Scholarships and alumni funds	Colleges show alumni which students and needs their money supported

10.2 Illustrative scenario: cyclone relief in a coastal district

1. **Day 1:** an NGO creates a "Cyclone Relief" campaign on the blockchain with a target of ₹5,00,000.
2. **Days 1–7:** citizens donate; each donor receives a transaction hash as a permanent receipt.
3. **Day 8:** the NGO records ₹2,00,000 for food kits and ₹1,00,000 for medicine, each with a note.
4. **Day 10:** donors and journalists check the Transparency page and verify individual hashes.
5. **Audit:** auditors compare bills and receipts against the on-chain allocation records.

The result is that every rupee in and out is public, permanent and checkable without trusting a private spreadsheet. This scenario is illustrative; collecting real money would need a payment-gateway integration (Chapter 11).

Chapter 11: Advantages, Limitations, Future Scope and Conclusion

11.1 Advantages

- Donation and allocation records are public and tamper-evident.
- Anyone can verify any transaction using its hash, directly from the blockchain.
- Rules are enforced by code: spending can never exceed donations, and only the owner can write.
- Donors need no crypto wallet; they use a normal website and amounts in rupees.
- Spending is visible category-wise (Food, Medicine, Shelter, Transportation, Other).

11.2 Limitations

- Blockchain does not prove that the organization itself is genuine; that remains an off-chain, administrative check.
- Allocations are self-reported: the contract proves what the organization claimed and when, not that goods were delivered.
- Donors do not sign their own transactions, so they trust the backend to record their donation (it cannot be changed once recorded).
- No real payment gateway; rupee amounts are recorded, not transferred.
- Unique donors are counted by name, and donor names are public (an "Anonymous" option is provided).
- The local test blockchain is reset when the Hardhat node restarts.

11.3 Future scope

- Integrate UPI or a payment gateway and store the payment reference on-chain.
- Let donors who have a wallet donate directly with MetaMask and sign their own transactions.
- Upload bills and receipts to IPFS and store their hash with each allocation.
- Require multi-signature approval (for example, two of three trustees) for each allocation.
- Deploy on a public testnet or a low-fee network such as Polygon, with block-explorer links.
- Send donors their transaction hash automatically by SMS or email.

11.4 Conclusion

ReliefChain shows that a smart contract can make the finances of a relief campaign transparent, verifiable and rule-based. Every donation and allocation is recorded permanently, the contract rejects fake records and over-spending, and anyone can verify a transaction by its hash. The working prototype passed all 71 automated checks.

The project is also honest about what blockchain cannot do: **it does not automatically prove that a charity is genuine; it provides a transparent and tamper-evident record of donations and fund allocations.** Combined with off-chain verification of the organization, it can substantially increase donor trust.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
bitcoin.org/bitcoin.pdf

2. Ethereum Foundation, "Ethereum Whitepaper" and developer documentation. ethereum.org
3. Solidity documentation, version 0.8. docs.soliditylang.org
4. Hardhat documentation. hardhat.org
5. OpenZeppelin Contracts 5.x documentation (Ownable). docs.openzeppelin.com
6. web3.py documentation. web3py.readthedocs.io
7. FastAPI documentation. fastapi.tiangolo.com
8. React documentation. react.dev
9. World Food Programme, "Building Blocks." innovation.wfp.org
10. UNICEF, "UNICEF launches Cryptocurrency Fund," press release, 9 October 2019. unicef.org
11. Payments Journal, "Bitcoin nonprofit BitGive launches GiveTrack beta at Money20/20," 2017. paymentsjournal.com

Appendix A: Installation and Running

1. Install dependencies: `npm install` in the project folder, `npm install` in `frontend`, and in `backend` create a virtual environment and run `pip install -r requirements.txt`; copy `.env.example` to `.env`.
2. Terminal 1: `npx hardhat node` (keep it running).
3. Terminal 2: `npx hardhat test` (15 passing), then `npx hardhat run scripts/deploy.js --network localhost`.
4. Terminal 3: in `backend`, activate the virtual environment and run `uvicorn main:app --reload --port 8010`.
5. Terminal 4: in `frontend`, run `npm run dev`, then open `http://localhost:5180`.
6. Admin login key: `reliefchain-admin` (set as `ADMIN_KEY` in `backend/.env`).
7. After any restart of the Hardhat node, run the deploy command again.